

# 面向流式点云识别的固定状态可加 FWP 网络

Dian Yu

University of New South Wales, Sydney, NSW 2052, Australia

dian.yu2@student.unsw.edu.au

ORCID: <https://orcid.org/0009-0002-0107-1014>

中文对照校对稿（对应 The Visual Computer 投稿版 main.tex）

## 摘要

多数点云网络通过重新访问已保存的点、重建邻域或执行全局最大池化来获得上下文。这些操作适合离线处理，却使持续增长的点流依赖完整历史。PointProgrammerNet 用三个固定的  $128 \times 170$  矩阵取代点历史。每个观测点产生一个表示“写到哪里”的归一化空间地址，以及一个表示“写入什么”的值向量；二者的外积直接加到矩阵中。这一简单构造使独立编码的分块可以通过矩阵加法精确合并，同时也支持相减和指数衰减。分割任务在指定坐标读取矩阵，分类任务则在删除全部输入点后仅使用矩阵。本文证明输入顺序不变性、分块与分布式编码等价性、固定保留内存，以及输出对新增证据的连续依赖。使用 XYZ 输入时，模型在 ShapeNetPart 上取得 92.95% 点准确率、82.58% 实例 mIoU 和 79.10% 类别 mIoU，在 ModelNet40 上取得 86.59% 分类准确率。三个 float32 状态始终占用 255 KiB，与点流长度无关。当累计点数达到 8,192 时，只更新新分块比重建完整状态快  $6.06\times$ ，相对状态误差仍低于  $10^{-6}$ 。这些结果展示了用有界、可直接更新与合并的状态取代不断增长的点历史时，所付出的精度代价与获得的实际收益。

**关键词：**点云；流式学习；可加状态；快速权重编程器；分割；分类

## 1 引言

点云是一个无序集合，但传感器是随时间观测它的。帧内处理应当与点的顺序无关，跨帧处理则不应要求重放完整历史。PointNet [1] 通过对称最大池化获得全局描述子，PointNet++ [2] 构建多尺度度量邻域。两者在离线场景下都很有效。然而最大值不是一个可加状态：固定摘要通常无法通过简单相加来支持删除、独立分块合并以及带连续权重的证据累积。

本文研究一个更严格的设定：一个点在写入之后可以立即被丢弃。后续计算只访问固定矩阵，分割任务额外接收一个查询坐标。一个学习到的键把每个点分配到各个状态行上，一个二阶值记录质量、位置、学习与交叉矩。把这些逐点写入相加，就得到固定尺寸的状态。

因此，独立分块可以被合并，已保存的子集可以被减去，旧证据可以被衰减。新点更新状态时不需要再遍历此前的观测。同样的代数支持分布式感知：边缘设备编码不相交的区域或时间窗，传输固定矩阵，中心节点在分类或分割之前把这些矩阵相加。分割读取一个由坐标条件化的场，不保存任何历史点特征；分类只读取状态行，并丢弃完整点表。

本文的贡献是：

- 一个三尺度二阶快速权重编程器 (FWP) 状态, 其内存不随观测点数增长;
- 可加的逐点写入, 支持精确的流式处理、层次化合并与分布式聚合, 且无需重放分块内容, 同时允许写入与读出完全并行;
- 一个坐标条件化的分割读出头与一个仅依赖状态的分类读出头, 两者都不对解码后的点做池化;
- 覆盖可加性、状态运算、精度与更新代价的证明与实验, 其中包含一个因果实验, 表明在参数量相同的前提下三个并行状态优于单一状态。

## 2 相关工作

把无序点集压缩为固定摘要, 已有若干成熟做法。它们的主要差别在于: 在一个点被吸收的那一刻, 各自放弃了什么。这个取舍决定了此后还有哪些状态运算可用。我们按这一取舍对相关工作分组, 然后说明本架构保留了哪些性质。

### 2.1 对称池化与度量邻域

PointNet 先做逐点变换再做最大池化 [1], PointNet++ 在此基础上加入层次化度量邻域 [2], Deep Sets 刻画了由求和构建的置换不变函数 [3]。三者都与点序无关, 且 Deep Sets 使用的求和在式 (14) 的严格意义下是可加的。但不断增长的点流会撞上两处障碍。逐通道最大值丢弃了撤销它所需的信息, 某个观测一旦被移除, 已保存的摘要就无法再被修正。度量邻域是相对于当前持有的点定义的; 插入或删除一个点, 其他点所属的邻域也随之改变, 分组只能重建。Deep Sets 绕开了这两处, 代价是求和进一个单一的全局向量, 它不携带空间地址, 无法在某个被请求的坐标上求值。我们保留可加的求和, 但把它铺展到 128 个带空间地址的行上, 使坐标条件化的读取成为可能, 同时不放弃可加性。

### 2.2 基于学习中心的固定尺寸摘要

VLAD 把局部描述子分配到学习到的中心并累加残差, 从而压缩表示 [4]。NetVLAD 把该分配变为软分配且可微 [5], PointNetVLAD 把这一构造用于点云的地点识别 [6]。其累加器与我们的形状相同: 一个由软分配到学习中心而构建的固定矩阵。差别在于累加什么, 以及之后又对它做了什么。被累加的残差是一阶的, 而式 (3) 保留了直到二阶的全部矩; 分配是在特征空间中进行的, 而我们的地址是坐标的归一化函数, 对查询位置保持连续。决定性的差别在最后一步。NetVLAD 先对每个块做块内归一化, 再对展平后的描述子做  $L_2$  归一化, 而这两步都不是可加的——把两个部分描述子各自归一化后再相加, 并不能复现二者并集的描述子。原始累加器本身是可加的, 但已发表的构造把这一性质归一化掉了, 且从未使用它。我们让累加器保持未归一化, 把归一化推迟到读出时刻, 命题 1 因此得以成立。

### 2.3 对固定隐变量集合的注意力

Set Transformer 使用诱导点与注意力对集合作摘要 [7]。Perceiver IO 维护一个尺寸与输入无关的隐变量数组, 并通过一组输出查询对该隐变量做注意力, 从而解码任意输出, 包括逐

元素的稠密预测 [8]。「用查询读取固定隐变量以产生稠密输出」这一想法属于他们，而不属于本文。差别在编码端。softmax 交叉注意力在观测元素上做归一化，因此由两个不相交分块得到的隐变量并不等于两个隐变量之和；该表示无法通过算术进行合并、相减或衰减，原文也未主张任何此类性质。我们的写入不在观测集合上施加任何归一化，因此状态始终是一组彼此独立的逐点项之和。查询的含义也不同：Perceiver 的查询是学习到的位置嵌入，而我们的查询是与数据处于同一空间中的坐标，因此同一个状态可以在从未被观测过的位置上求值。

## 2.4 有界的循环状态

核化注意力可以用固定尺寸状态以循环方式求值 [9]，快速权重编程器把这类更新表达为可加外积或 delta 修正 [10]。这些正是我们的写入所使用的机制。近期点云工作也通过结构化状态空间模型采用有界状态；PointMamba 在循环之前用空间填充曲线把点序列化 [11]。序列化使隐状态成为遍历顺序的函数，因此它不是置换不变的，两个部分状态也无法相加。我们保留有界状态，但不保留序列，因为正是与顺序无关的求和换来了分块与设备等价性。

## 2.5 本架构拒绝做的事

上述每一族都为换取某项性质而交出了另一项：最大值交出可逆性，度量邻域交出编辑的局部性，归一化描述子交出可加性，softmax 编码器交出可分解性，序列化的循环交出顺序不变性。本文的设计源自写入时刻的一条拒绝：任何把某个点的写入与另一个点耦合起来的東西，都不得进入持久状态。这排除了在观测集合上的归一化、点与点之间的比较、选择与排序。聚合方法通常在吸收一个点时所做的运算——密度归一化、相对参考位置的中心化，以及非线性解释——在这里都被推迟到读出，从而不再触及所存矩阵的代数。表 1 给出由此确定的位置。

这一拒绝带来了什么、又付出了什么，都是被测量出来的，而不是被断言的。第 4.2 节隔出并行状态结构的贡献，表 3 与表 4 则把由此得到的精度置于层次化基线之旁。

我们在 ModelNet40 [12] 上评估分类，在 ShapeNetPart [13] 上评估部件分割，后者派生自 ShapeNet [14]。

# 3 方法

设  $X = \{\mathbf{x}_i\}_{i=1}^N$ ，归一化坐标  $\mathbf{x}_i \in \mathbb{R}^3$ 。PointProgrammerNet 用三个矩阵  $\mathbf{S}_s \in \mathbb{R}^{128 \times 170}$  取代不断增长的点历史，其中  $s \in \{f, m, b\}$  表示初始半径分别为 0.08、0.20 和 0.60 的三个分支。半径、空间中心、点编码器与读出网络都独立学习。状态是为每个物体或点流创建的，与训练得到的模型参数相互独立。

三个分支的状态尺寸相同，使用相同的方程。不同的初始半径只提供不同的起始条件；每个分支最终表示什么由训练决定。分支首先用一个两层点网络把坐标映射为学习内容，

$$\mathbf{g}_s(\mathbf{x}) = \text{ReLU}(\text{BN}(W_{s2} \text{ReLU}(\text{BN}(W_{s1}\mathbf{x} + \mathbf{b}_{s1})) + \mathbf{b}_{s2})) \in \mathbb{R}^{32}. \quad (1)$$

矩阵  $W_{s1}, W_{s2}$  与偏置在每个分支中独立学习，两个隐藏层宽度均为 32。因此状态尺寸相同并不意味着特征重复。该非线性映射对每个点独立求值，所以它丰富了一个点所写入的内容，却

表 1: 各族集合摘要所保留的性质。「顺序无关写入」指摘要不依赖点被吸收的顺序;「可加状态」指并集的摘要等于各摘要之和;「有界状态」指保留内存不随观测点数增长;「坐标读取」指仅凭摘要即可在被请求的位置上求值。

| 族                         | 顺序无关<br>写入     | 可加<br>状态       | 有界<br>状态 | 坐标<br>读取       |
|---------------------------|----------------|----------------|----------|----------------|
| PointNet, 最大池化            | 是              | 否              | 是        | 否              |
| PointNet++, 度量邻域          | 是              | 否              | 否        | 否              |
| Deep Sets, 全局求和           | 是              | 是              | 是        | 否              |
| VLAD、NetVLAD、PointNetVLAD | 是              | 否 <sup>a</sup> | 是        | 否              |
| Set Transformer, 诱导点      | 是              | 否 <sup>b</sup> | 是        | 否              |
| Perceiver IO, 输出查询        | 是              | 否 <sup>b</sup> | 是        | 是 <sup>c</sup> |
| 线性注意力、快速权重                | 是 <sup>d</sup> | 是              | 是        | 否              |
| 序列化点上的状态空间模型              | 否              | 否              | 是        | 否              |
| PointProgrammerNet (本文)   | 是              | 是              | 是        | 是              |

<sup>a</sup>原始累加器是可加的, 但块内归一化与  $L_2$  归一化移除了该性质。<sup>b</sup>在观测元素上的 softmax 把它们耦合在一起。<sup>c</sup>Perceiver IO 从学习到的位置查询解码稠密输出, 而非从输入空间中的坐标。<sup>d</sup>外积更新本身与顺序无关, 尽管这些模型被用于有序序列。

没有把这次写入与任何其他点耦合; 可加性得以保持, 是因为持久状态中累加的只有随后的外积。

### 3.1 可加状态的构造

128 个状态行各有一个学习到的中心  $\mathbf{c}_{sj} \in \mathbb{R}^3$ 。这些中心不是从输入点云中采样的。训练之前, 用一个 Sobol 序列在  $[-0.9, 0.9]^3$  内放置 128 个初始锚点, 使各行一开始就有较宽的空间覆盖。每个分支拥有自己可训练的一份副本, 训练过程可以通过有界参数化  $\mathbf{c}_{sj} = 0.95 \tanh(\boldsymbol{\theta}_{sj})$  移动这些锚点。对一个观测坐标  $\mathbf{x}$ , 分支计算它到全部 128 个锚点的距离, 并把这些距离转换为 128 个连续的写入份额。正半径  $r_s = \exp(-\lambda_s)$  控制这些份额的集中程度。归一化地址为

$$k_{sj}(\mathbf{x}) = \frac{\exp[-\|\mathbf{x} - \mathbf{c}_{sj}\|^2/(2r_s^2)]}{\sum_{u=1}^{128} \exp[-\|\mathbf{x} - \mathbf{c}_{su}\|^2/(2r_s^2)]}. \quad (2)$$

分母把每个点的总写入质量固定为 1。这避免了稠密分块改变单次写入的尺度, 也使半径只控制集中度而不控制总量。直观地说, 这些中心相当于 128 个软空间箱。靠近若干中心的点会给这些行较大份额、给其余行很小份额; 它绝不会只被分配给一个箱。地址决定点写到哪里, 而下面这个 170 维的值决定它贡献哪些统计量:

$$\mathbf{v}_s(\mathbf{x}) = [1, \mathbf{x}, \mathbf{g}_s, \text{vec}(\mathbf{x}\mathbf{g}_s^\top), x^2, y^2, z^2, xy, xz, yz, \mathbf{g}_s \odot \mathbf{g}_s]. \quad (3)$$

各块维度为  $1 + 3 + 32 + 96 + 6 + 32 = 170$ , 分别保留质量、一阶矩、学习内容、坐标一内容乘积与二阶矩。开头的 1 之后用作局部平均的分母。六个坐标二次项与 32 个特征平方项保留了均值无法恢复的离散程度。等价地说, 值向量收集了  $(\mathbf{x}, \mathbf{g}_s)$  直到二阶的全部矩, 只是把特征一特征块限制在其对角线上, 使其宽度对内容维度保持线性。

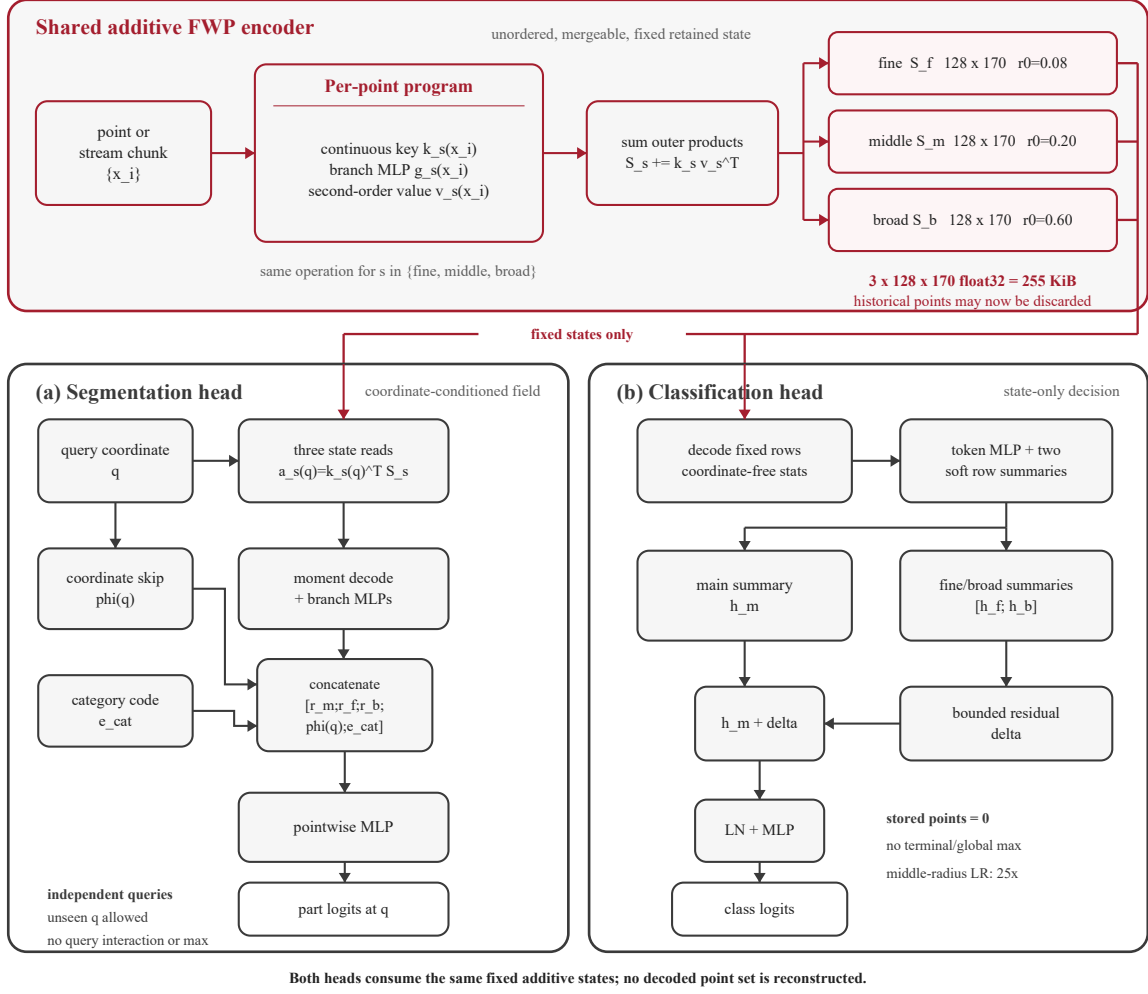


图 1: PointProgrammerNet 架构。红色路径承载三个等尺寸可加 FWP 状态，此时历史点已被丢弃。分割头在查询坐标  $q$  处读取每个状态，然后拼接解码得到的上下文、坐标跳连与类别编码。分类头解码固定行，并把一个有界的“细一粗”残差加到中尺度摘要上。两个读出头都不重建也不池化任何解码点集。

一个点对状态行  $j$  的改变为

$$[\Delta S_s(\mathbf{x})]_{j,:} = k_{sj}(\mathbf{x}) \mathbf{v}_s(\mathbf{x})^\top. \quad (4)$$

这一秩一更新使用的正是 FWP 的定义性操作：键选择写到哪里，值提供写入什么，独立的写入通过相加叠加 [10]。PointProgrammerNet 保持了这种线性叠加，既没有在观测点集上做写入归一化，也没有对先前状态做非线性覆写。因此一行是一个加权累加器，而不是某个被选中点的槽位；把这些外积相加得到

$$\mathbf{S}_s(X) = \sum_{i=1}^N \mathbf{k}_s(\mathbf{x}_i) \mathbf{v}_s(\mathbf{x}_i)^\top = \mathbf{K}_s^\top \mathbf{V}_s. \quad (5)$$

没有点被分配给单一行，也没有点与另一个点作比较。一旦它的三个外积被加入，它就可以被释放。

表 2: 单个点写入的值。每个状态行都有这 170 列。

| 块                                 | 维度 | 保留的信息   |
|-----------------------------------|----|---------|
| 1                                 | 1  | 累计质量    |
| $\mathbf{x}$                      | 3  | 坐标和     |
| $\mathbf{g}_s$                    | 32 | 学习内容和   |
| $\mathbf{x}\mathbf{g}_s^\top$     | 96 | 位置-内容关系 |
| $\mathbf{x}\mathbf{x}^\top$       | 6  | 几何二阶矩   |
| $\mathbf{g}_s \odot \mathbf{g}_s$ | 32 | 特征二阶矩   |

对新到达的分块,  $\mathbf{K}_s \in \mathbb{R}^{M \times 128}$  堆叠其地址,  $\mathbf{V}_s \in \mathbb{R}^{M \times 170}$  堆叠其值。分块更新就是一次矩阵乘积  $\Delta \mathbf{S}_s = \mathbf{K}_s^\top \mathbf{V}_s$ 。持久状态按  $\mathbf{S}_s \leftarrow \mathbf{S}_s + \Delta \mathbf{S}_s$  更新, 此后  $\mathbf{K}_s$ 、 $\mathbf{V}_s$  以及分块中的点都可以丢弃。分块大小只改变临时工作内存, 不改变保留下来的表示。

因此完整的流式流程很短。对每个到达的分块, 编码器 (i) 在共享参考系中归一化其坐标, (ii) 计算三组地址与值矩阵, (iii) 把三个乘积  $\mathbf{K}_s^\top \mathbf{V}_s$  加到持久状态上, (iv) 释放全部与分块相关的张量。独立设备执行同样的四步, 并只传输它们的三个矩阵。推理时, 分割头只在被请求的坐标上求值, 分类头则直接消费这些矩阵。后续任何阶段都不需要从状态重新生成点表。

### 3.2 任务读出

在分割中,  $\mathbf{x}_i$  表示一个已经被加入状态、随后可被丢弃的观测点;  $\mathbf{q}$  仅仅是请求标签的那个坐标。在 ShapeNetPart 上, 查询使用带标注的输入坐标, 但解码器接收的是坐标与固定状态, 而不是保存下来的点特征。查询通过式 (2) 产生地址并读取

$$\mathbf{a}_s(\mathbf{q}) = \mathbf{k}_s(\mathbf{q})^\top \mathbf{S}_s = \sum_i \underbrace{\mathbf{k}_s(\mathbf{q})^\top \mathbf{k}_s(\mathbf{x}_i)}_{w_{si}(\mathbf{q})} \mathbf{v}_s(\mathbf{x}_i)^\top. \quad (6)$$

系数  $w_{si}(\mathbf{q})$  是对该矩阵读取的代数解释: 当点  $i$  强烈写入了  $\mathbf{q}$  强烈读取的那些行时, 它就大。实现直接计算左端的矩阵乘积; 它既不重建也不重访点  $i$ , 也不对解码点做全局最大池化。

$\mathbf{a}_s(\mathbf{q})$  的第一个分量是读取质量  $\mu_s(\mathbf{q})$ 。实现通过下式避免在近乎空白区域出现不稳定的除法:

$$\epsilon_s = 10^{-3} \text{mean}_{\mathbf{q}'} \mu_s(\mathbf{q}') + 10^{-20}, \quad \tilde{\mu}_s(\mathbf{q}) = \max(\mu_s(\mathbf{q}), \epsilon_s). \quad (7)$$

其中的均值在单个样本的查询坐标上取, 并从梯度中分离。式 (7) 中的  $\max$  只是一个标量数值下限; 它既不比较点特征, 也不执行全局最大池化。把保存的坐标、特征、交叉乘积与平方和除以  $\tilde{\mu}_s$ , 得到局部坐标均值与特征均值  $\bar{\mathbf{x}}_s, \bar{\mathbf{g}}_s$ 、交叉矩  $E_{xg,s}$ 、几何矩  $E_{xx,s}$  与特征能量  $E_{gg,s}$ 。它们构成 170 维描述子

$$\mathbf{d}_s(\mathbf{q}) = [\bar{\mathbf{g}}_s, \bar{\mathbf{x}}_s - \mathbf{q}, \log \tilde{\mu}_s, \text{vec}(E_{xg,s} - \mathbf{q}\bar{\mathbf{g}}_s^\top), \text{vech}(E_{xx,s} - \bar{\mathbf{x}}_s\bar{\mathbf{x}}_s^\top), \sqrt{[E_{gg,s} - \bar{\mathbf{g}}_s \odot \bar{\mathbf{g}}_s]_+ + 10^{-8}}]. \quad (8)$$

其中  $\text{vech}$  保留对称  $3 \times 3$  矩阵的六个不同元素,  $[\cdot]_+$  截去由舍入误差造成的负方差。其块维度为 32, 3, 1, 96, 6, 32, 同样合计 170。该描述子刻意组合了互补的统计量: 特征均值提供局部

语义内容，相对质心与几何协方差描述局部位置与形状，对数质量记录有多少证据支撑这次读取；坐标—特征项保留了内容在查询点周围如何变化，特征标准差则暴露出仅凭均值会被掩盖的混合与边界变化。每个分支使用同一个  $170 \rightarrow 160 \rightarrow 160$  的 BatchNorm-ReLU 解码器。最终的逐点头为

$$\mathbf{z}_{\text{seg}}(\mathbf{q}) = h_{\text{seg}}([\mathbf{r}_f(\mathbf{q}); \mathbf{r}_m(\mathbf{q}); \mathbf{r}_b(\mathbf{q}); \phi(\mathbf{q}); \mathbf{e}_{\text{cat}}]), \quad (9)$$

其中  $\mathbf{r}_s \in \mathbb{R}^{160}$  是一次解码后的状态读取， $\phi(\mathbf{q}) \in \mathbb{R}^{32}$  是坐标特征， $\mathbf{e}_{\text{cat}} \in \mathbb{R}^{16}$  是 ShapeNet-Part 的类别编码。分割 MLP 为  $528 \rightarrow 256 \rightarrow 128 \rightarrow 50$ ，前两个仿射层之后带 BatchNorm、ReLU 与 dropout 0.3。坐标通路是绕过状态压缩的类 U-Net 跳连。该头对每个查询单独作用，从不在解码点之间做池化。历史特征可以删除，只有之后要被标注的坐标必须保留。同一个固定状态也可以在未被观测过的坐标上查询。

在分类中不保留任何坐标。对第  $j$  行，保存的 170 个元素按下式划分：

$$[\mu_{sj}, \mathbf{p}_{sj}, \mathbf{f}_{sj}, \text{vec}(\mathbf{C}_{sj}), \text{vech}(\mathbf{Q}_{sj}), \mathbf{u}_{sj}],$$

即质量、坐标和、特征和、坐标—特征和、对称坐标二阶矩与特征平方和。行归一化使用

$$\tilde{\mu}_{sj} = \max\left(\mu_{sj}, 10^{-4} R^{-1} \sum_{u=1}^R |\mu_{su}| + 10^{-20}\right), \quad (10)$$

其下界在反向传播中被分离。记  $\bar{\mu}_s = R^{-1} \sum_u \tilde{\mu}_{su}$ ，除以  $\tilde{\mu}_{sj}$  得到

$$\begin{aligned} \mathbf{d}_{sj}^{\text{row}} &= [\bar{\mathbf{g}}_{sj}, \bar{\mathbf{x}}_{sj}, \log(\tilde{\mu}_{sj}/\bar{\mu}_s), \text{vech}(\text{Cov}_{xx,sj}), \\ &\quad \text{Std}_{g,sj}, \text{vec}(\text{Cov}_{xg,sj})] \in \mathbb{R}^{170}, \end{aligned} \quad (11)$$

块维度为 32, 3, 1, 6, 32, 96。一个独立的  $170 \rightarrow 128 \rightarrow 128$  LayerNorm-GELU 网络编码每一行，得到  $\mathbf{T}_s \in \mathbb{R}^{128 \times 128}$ 。两列学习到的打分对固定行做摘要：

$$\mathbf{U}_s = \text{softmax}_{\text{rows}}(\text{score}_s(\mathbf{T}_s)), \quad \mathbf{h}_s = H_s(\text{vec}(\mathbf{U}_s^\top \mathbf{T}_s)) \in \mathbb{R}^{256}. \quad (12)$$

这个 softmax 作用在 128 个保留的状态行上，而不是作用在输入点上，因此编码之后其代价是固定的。中尺度摘要作为参考，另外两个构成有界残差：

$$\begin{aligned} \boldsymbol{\delta} &= \sigma(a) \tanh(W[\mathbf{h}_f; \mathbf{h}_b] + \mathbf{b}), \\ \mathbf{z}_{\text{cls}} &= h_{\text{cls}}(\text{LN}(\mathbf{h}_m + \boldsymbol{\delta})). \end{aligned} \quad (13)$$

残差投影从零开始， $\sigma(a)$  从 0.1 开始。分类器  $h_{\text{cls}}$  为  $256 \rightarrow 256 \rightarrow 40$ ，带 GELU 与 dropout 0.3。所有半径都可学习；只有中尺度半径使用  $25\times$  学习率倍数与零权重衰减。分类器只使用固定矩阵，末端没有任何点最大池化。两列打分为每个状态提供互补的、代价固定的视角，而不是把所有行都压成一个平均。以中尺度分支作为初始稳定的参考、并让细尺度与粗尺度摘要经由一个零起点的有界残差进入，可以避免三个分支在各自学到有用摘要之前互相干扰；当辅助证据确实有助于分类时，该残差随后可以增大。

### 3.3 流式性质

对任意点多重集  $A$  与  $B$ , 式 (5) 给出

$$\mathbf{S}_s(A \uplus B) = \mathbf{S}_s(A) + \mathbf{S}_s(B). \quad (14)$$

有限和同时也使状态与点的顺序无关。因此, 分别编码的分块、设备或聚合树的分枝都可以通过相加合并。若可移除子集的状态可用, 就可以将其减去; 指数遗忘使用  $\mathbf{S}_{s,t} = \rho \mathbf{S}_{s,t-1} + \mathbf{S}_s(X_t)$ 。精确的分布式等价性要求共同的训练权重、坐标归一化与参考系。

记  $F_s(\mathbf{x}) = \mathbf{k}_s(\mathbf{x})\mathbf{v}_s(\mathbf{x})^\top$  为单个点的秩一 FWP 写入, 于是  $\mathbf{S}_s(X) = \sum_{\mathbf{x} \in X} F_s(\mathbf{x})$ 。

**命题 1** (顺序、分块与设备). 对点多重集  $A, B$ , 有  $\mathbf{S}_s(A \uplus B) = \mathbf{S}_s(A) + \mathbf{S}_s(B)$ , 且状态与点的顺序无关。使用相同训练编码器、坐标归一化与参考系的设备, 因而可以通过把各自独立编码的状态相加而得到集中式状态。

**证明.** 两端展开为同一组有限的逐点矩阵  $F_s(\mathbf{x})$ 。在实数运算下, 重排或重新分组一个有限和不改变其值。float32 结果的差异仅来自累加顺序。□

为把分布式主张写清楚, 设设备  $d$  观测点多重集  $X^{(d)}$ , 且所有设备使用同一个编码器。中心节点构造

$$\mathbf{S}_s^{\text{global}} = \sum_{d=1}^D \mathbf{S}_s(X^{(d)}) = \mathbf{S}_s\left(\biguplus_{d=1}^D X^{(d)}\right). \quad (15)$$

该等式由两端展开为同一组逐点外积之和得到。无论部分状态是同时到达、异步到达, 还是经过一棵中间聚合树, 它都成立。分类随后只使用合并后的矩阵。分割则额外接收待查询的坐标与物体类别编码。原始点特征无需离开感知设备。

同样的代数在所需部分状态可用时给出可逆的状态运算。若  $Y$  是一个已知子集, 则  $\mathbf{S}_s(X \setminus Y) = \mathbf{S}_s(X) - \mathbf{S}_s(Y)$ 。这在不重放  $X$  的情况下移除了  $Y$  的聚合贡献。在追加之前乘以  $\rho \in [0, 1]$  即实现指数遗忘。这些陈述针对表示本身; 当某子集的独立状态未被保留时, 相减无法从状态中恢复其中的单个点。

**命题 2** (保留内存与更新代价). 若每个分支有  $R_s$  行、 $D_s$  列, 则保留内存为  $O(\sum_s R_s D_s)$ , 与历史长度无关。编码  $M$  个新点的代价为  $O(M \sum_s R_s D_s)$ , 且不重访更早的观测。

**证明.** 只有固定矩阵是持久的。无论此前有多少分块, 一个新分块都只贡献同样维度的新外积。□

为刻画一个逐渐到达的观测, 用证据强度  $t \in [0, 1]$  缩放其写入:

$$\mathbf{S}_s(t) = \mathbf{S}_s^{\text{old}} + t \mathbf{k}_s(\mathbf{x})\mathbf{v}_s(\mathbf{x})^\top. \quad (16)$$

状态对  $t$  是仿射的。高斯地址、softmax、带正下限的归一化矩读取、仿射层、BatchNorm 或 LayerNorm、ReLU 或 GELU, 以及稳定化的平方根都是连续的。它们的复合使分类 logits 与固定坐标下的分割 logits 对  $t$  连续且分段光滑。空间地址对查询坐标  $\mathbf{q}$  也是连续的, 因此一个固定状态定义了一个连续、分段光滑的分割场。这并不意味着硬 argmax 标签会连续变化: 当两个 logits 交叉时它仍可跳变。它也不能建立校准性或“证据越多必然越好”的单调性。



对已发布的配置，三个分支的  $R_s = 128$ 、 $D_s = 170$ ，因此持久表示包含 65,280 个数。以 float32 计为每个点流 261,120 字节（255 KiB）。临时张量随到达分块的大小变化，可以立即释放。分割应用可以保留一小组感兴趣的坐标，但这些坐标是外部查询，而不是历史状态的一部分；分类应用则完全不保留坐标。

可加规则把所有帧视为可交换的证据。当帧的顺序本身应当改变记忆时，未来的扩展可以采用帧级的快速权重 delta 更新 [10]:

$$\mathbf{S}_{s,t} = \rho \mathbf{S}_{s,t-1} + \eta_t \mathbf{K}_{s,t}^\top (\mathbf{V}_{s,t} - \mathbf{K}_{s,t} \mathbf{S}_{s,t-1}). \quad (17)$$

其中  $\mathbf{K}_{s,t}$  与  $\mathbf{V}_{s,t}$  打包第  $t$  帧中的全部键与值， $\rho$  保留旧状态， $\eta_t$  是更新率。在帧内重排点相当于对两个矩阵同时左乘同一个置换  $P$ ；由于  $P^\top P = I$ ，修正项不变。因此该扩展保持帧内点序不变性，同时引入帧间有序的状态转移。它是一个提出的方向，未用于本文报告的实验，因为它有意放弃了跨帧的精确可加性。

### 3.4 并行求值

第 2.5 节的那条拒绝，同时也决定了计算可以如何铺开。在一个分块内部，某个点的写入不读取任何其他点，因此全部  $M$  个点被同时求值，分块更新就是式 (5) 的一次矩阵乘积。在分块与设备之间，命题 1 使合并成为求和，而求和满足结合律与交换律，因此部分状态可以并发产生并以任意次序归约，包括对  $D$  个产生者做深度为  $\log D$  的树形归约，其间无需同步、也无需固定调度。三个分支在写入过程中不共享任何项，它们的地址与值构成三个彼此独立的矩阵乘积，可以并发发起或批量处理。读出的情形相同：分割头对每个查询单独作用，从不在解码查询之间做池化，因此一组被请求的坐标同样是一次矩阵乘积加逐点网络，这些查询可以在设备之间划分而无需任何交换。

有一个细节使其中第一条成为可能。式 (2) 中的地址确实做了归一化，但归一化的是本分支自己的 128 个行，而不是观测集合，因此分母始终留在单个点自己的计算之内，从不变成跨点的归约。编码一个分块的关键路径于是不随其中点数增长，这也正是第 4.3 节中实测的增量时延保持平坦、而重建时间随历史增长的原因。

有几种常见的构造不允许这种铺开方式。度量邻域需要在当前持有的点上做依赖数据的搜索，点集必须驻留，且访存是不规则的。对序列化后的点做循环，在序列维度上是顺序执行的，其结果依赖遍历方式，因此无法独立产生部分结果再合并。像 softmax 注意力那样在观测集合上取归一化分母，本身就是一次跨点归约，也就成了写入内部的一个同步点。

## 4 实验

### 4.1 协议与精度

我们把每个物体居中，并将 XYZ 除以其最大绝对坐标，使用 1,024 个点。ShapeNetPart 含 13,972 个训练形状与 2,874 个测试形状；ModelNet40 含 9,840 与 2,468 个。模型用 AdamW 训练 20 个 epoch，基础学习率  $3 \times 10^{-3}$ ，权重衰减  $10^{-4}$ ，余弦衰减，批大小分别为 16 与 32。分类使用 0.1 的标签平滑。结果为种子 0、1、2 上的均值  $\pm$  总体标准差。分割模型含 341,909 个可训练参数，纯状态分类器含 529,426 个。在分类中，只有中尺度半径使用  $25 \times$  学

表 3: ShapeNetPart 分割 (XYZ 输入)。PointProgrammerNet 从不在解码查询点上做池化。

| 模型                 | 点准确率 (%)         | 实例 mIoU (%)      | 类别 mIoU (%)      | 参数量     |
|--------------------|------------------|------------------|------------------|---------|
| 轻量 PointNet++      | $93.41 \pm 0.05$ | $83.88 \pm 0.11$ | $79.51 \pm 0.24$ | 573,106 |
| PointProgrammerNet | $92.95 \pm 0.02$ | $82.58 \pm 0.15$ | $79.10 \pm 0.25$ | 341,909 |

表 4: ModelNet40 分类 (1,024 点)。Density 指固定的密度梯度偏移。

| 模型                 | 干净 (%)           | 密度 (%)           | 掉分 (pp)         | 参数量     |
|--------------------|------------------|------------------|-----------------|---------|
| 轻量 PointNet++      | $89.84 \pm 0.43$ | $85.47 \pm 0.88$ | $4.38 \pm 1.18$ | 207,784 |
| 参数对齐的 PointNet     | $87.90 \pm 0.27$ | $87.55 \pm 0.38$ | $0.35 \pm 0.12$ | 442,838 |
| PointProgrammerNet | $86.59 \pm 0.23$ | $82.47 \pm 0.93$ | $4.12 \pm 0.71$ | 529,426 |

习率倍数且不加权重衰减；第 4.5 节的受控筛选说明了这一面向鲁棒性的选择。所报告的全部 PointProgrammerNet 变体都使用三个  $128 \times 170$  状态，且在输入点或解码点上都没有全局最大池化。我们的紧凑 PointNet++ 实现是本地受控基线，而非权威复现。

PointProgrammerNet 的点准确率比轻量 PointNet++ 对照低 0.46 个百分点，实例 mIoU 低 1.30 个百分点，而参数量少 40.3%。

纯状态分类器达到 86.59% 的干净准确率，密度偏移下为 82.47%。表 4 把它与两个对照并列，并量化了“丢弃点表、只从可合并状态分类”所付出的代价。

## 4.2 并行状态与单一状态

三个状态并非冗余，其收益也不是参数量效应。我们用一个严格配对的因果实验来验证：对照组保留完整模型及其初始化，但移除两路辅助读取，使得只有主状态能够影响输出。因此两侧的参数量与初始权重完全相同。表 5 报告了该筛选实验，训练 5 个 epoch，种子 0、1、2，使用 2,048 个训练形状与 512 个测试形状。

在参数量不变的前提下，移除两个辅助状态使实例 mIoU 下降 1.05 至 1.20 个百分点。改为给某一个状态增加容量同样无济于事：在保留三个状态的前提下把主状态从 128 行加倍到 256 行，只达到 74.79，仍低于任何一个所有状态均为 128 行的配置。已发布模型使用三个状态、341,909 参数，在同一筛选协议下达到 77.22，比单状态对照高出 2.54 个百分点，而参数量少 29.5%。并行状态所贡献的可用结构，既不是更大的参数预算能提供的，也不是加宽主状态能提供的。这些是在数据子集上的短程筛选，作为受控比较报告，不作为最终精度。

## 4.3 可加性与更新代价

在 32 个物体上，由 2–32 个有序或打乱分块构建的状态与一次性构建的结果一致，相对误差低于  $5.3 \times 10^{-7}$ 。追加、合并、移除与衰减测试均低于  $9.8 \times 10^{-7}$ 。三个 float32 状态含 65,280 个标量 (255 KiB)，与点流长度无关。对短点云而言原始 XYZ 更小——实测的交叉点约为 21,800 个观测——其收益在于有界的保留内存，以及在长流或分布式场景中可直接进行的状态运算。

表 5: 并行状态与单一状态的对比。5 个 epoch 的筛选实验，三个种子，2,048 训练形状与 512 测试形状。单状态对照保留完整模型与初始化，但移除两路辅助读取。

| 配置           | 状态数 | 参数量     | 实例 mIoU (%) | 类别 mIoU (%) |
|--------------|-----|---------|-------------|-------------|
| 仅主状态（因果对照）   | 1   | 484,914 | 74.68       | 70.50       |
| 主状态加宽至 256 行 | 3   | 485,298 | 74.79       | 70.53       |
| 并行状态，保守半径学习率 | 3   | 484,914 | 75.73       | 71.03       |
| 并行状态，更强残差混合  | 3   | 484,914 | 75.88       | 70.97       |
| 已发布的三状态模型    | 3   | 341,909 | 77.22       | 73.08       |

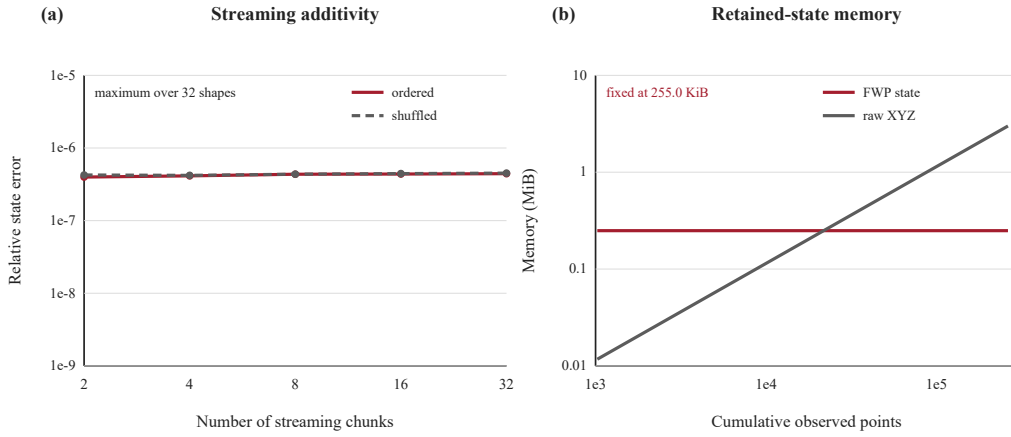


图 2: 可加流式处理与固定保留内存。有序与打乱的分块写入都能复现单批次 FWP 状态，误差仅为浮点累加误差。三个  $128 \times 170$  的 float32 状态始终占用 255 KiB，而保存 XYZ 坐标的开销随观测数线性增长。

在一块 RTX 3070 Ti Laptop GPU 上，四个点流每帧接收 256 个新点。增量时延稳定在 1.46–1.53 ms，而重建时延随历史增长。在累计 2,048、4,096 和 8,192 个点时，更新分别快 1.52 $\times$ 、3.04 $\times$  和 6.06 $\times$ 。在累计点数约 512 以下时，两条路径代价相当，重建甚至略快，因此该优势是渐近的而非普遍的。

#### 4.4 状态代数的实际验证

在 32 个 ShapeNetPart 物体上，把追加、独立编码后的合并、已保存帧的移除与指数衰减分别与直接重建作比较。它们的平均相对误差分别为  $2.11 \times 10^{-7}$ 、 $2.13 \times 10^{-7}$ 、 $3.03 \times 10^{-7}$  与  $1.77 \times 10^{-7}$ ，全部最大值低于  $9.8 \times 10^{-7}$ 。剩余差异与 float32 的累加顺序一致。经过 16 帧后，普通相加得到状态范数 14.51；取  $\rho = .9$  的衰减把范数限制在 7.42。该实验验证的是状态方程，并不主张时序任务上的精度。

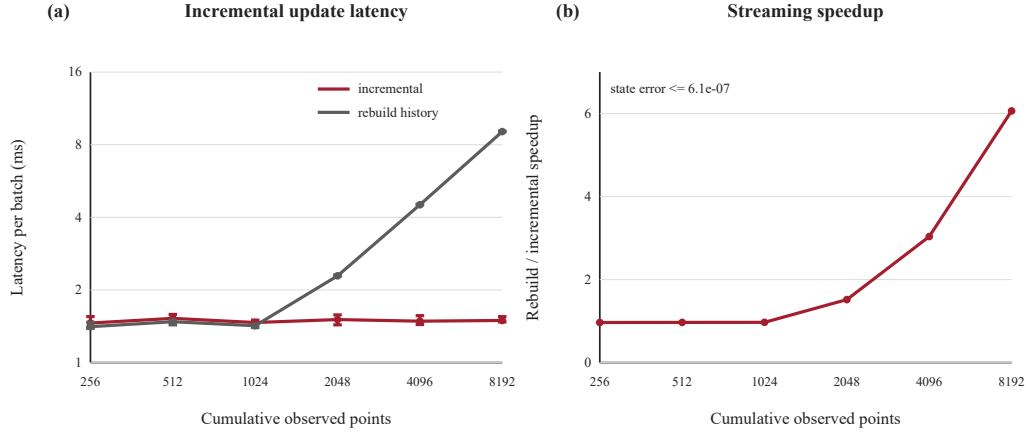


图 3: 设备端点流的 FWP 状态构建代价。(a) 增量更新只处理一个新的 256 点帧, 而重建要处理全部累计点。数值为四个点流、7 次交错试验、每次 30 遍重复的中位墙钟时延, 误差棒表示观测范围。(b) 加速比随历史长度增长, 同时增量状态与重建状态在数值上保持等价。

表 6: 5 个 epoch 的受控筛选。正值表示支持保留方案。

| 保留的方案                     | 配对的替代方案   | 配对变化                                      |
|---------------------------|-----------|-------------------------------------------|
| 分割: 等宽解码器                 | 中尺度解码器不等宽 | 实例 $+0.30 \pm 0.30$ ; 类别 $+0.15 \pm 0.29$ |
| 分割: 不同半径初值                | 相同半径初值    | 实例 $+0.74 \pm 0.42$ ; 类别 $-0.05 \pm 0.33$ |
| 分割: 统一归一化键                | 混合键定义     | 实例 $+0.35 \pm 0.16$ ; 类别 $+0.85 \pm 0.37$ |
| 分类: 中尺度残差                 | 直接拼接      | 干净 $+2.67 \pm 1.60$ ; 偏移 $+3.78 \pm 1.71$ |
| 分类: 中尺度半径 $25 \times$ 学习率 | 基础半径学习率   | 干净 $+0.13 \pm 1.03$ ; 偏移 $+0.91 \pm 0.33$ |

#### 4.5 最终读出头的受控筛选

这些筛选每次只隔离一个架构决策。所有筛选在同一比较内使用相同的数据划分与优化协议、5 个 epoch、种子 0/1/2, 以及 2,048 个训练形状与 512 个测试形状的子集。这些短程运行用于选择结构性方案; 正文主表报告的是在完整划分上 20 个 epoch 的最终结果。表 6 报告保留方案相对于所列替代方案的配对变化。

等宽的  $170 \rightarrow 160 \rightarrow 160$  分割解码器使实例 mIoU 提升  $0.30 \pm 0.30$  个百分点, 并减少 24,480 个参数。不同的半径初始化使实例 mIoU 提升  $0.74 \pm 0.42$ 。对三个分支使用同一个归一化地址方程, 相对混合键变体使实例与类别 mIoU 分别提升  $0.35 \pm 0.16$  与  $0.85 \pm 0.37$ 。对纯状态分类, 中尺度参考残差相对直接拼接使干净与密度偏移准确率分别提升  $2.67 \pm 1.60$  与  $3.78 \pm 1.71$ 。只提高中尺度半径的学习率带来的干净准确率变化不稳定, 但偏移准确率有稳定的  $0.91 \pm 0.33$  提升; 保留该设置正是为了这一稳定收益, 而非其干净准确率效应。这些结果支持了所发布的读出头, 同时使状态写入规则在三个分支间保持对称。

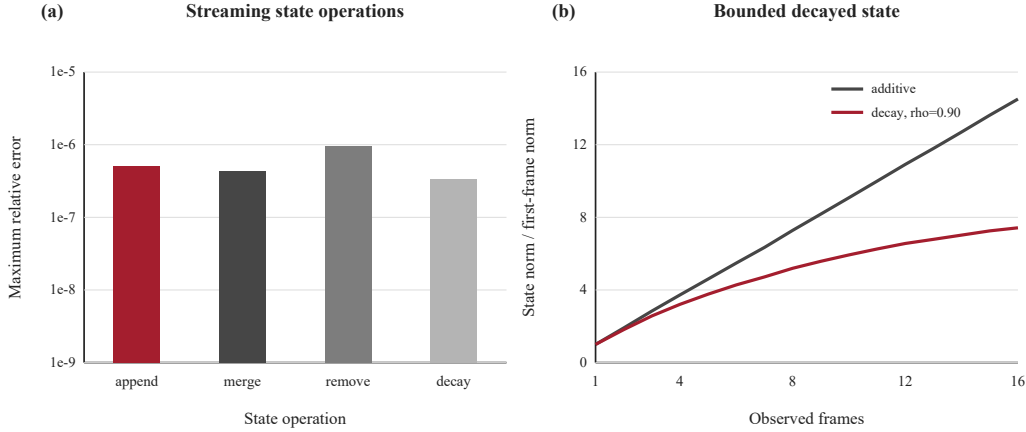


图 4: 流式状态运算。(a) 在 32 个 ShapeNetPart 物体上, 追加/合并、帧移除与指数衰减相对于独立重建的 FWP 状态的最大相对误差。(b) 普通相加使状态幅值随帧数增长, 而指数衰减在不改变状态尺寸的前提下限制了旧帧的影响。

## 5 讨论与局限

本文设定的场景是不断增长或分布式的点流。传感器可以编码一帧、合并其状态, 然后释放该帧。多台相机、激光雷达、机器人或边缘处理器可以在本地编码不相交的空间区域或时间窗, 并把固定状态发送到服务器。相加随后得到与集中式编码相同的表示, 因此聚合可以是异步且层次化的。原始点集无需离开感知设备。分割服务可以只保留它计划查询的坐标, 而分类器在更新证据时无需重放更早的点。上述任何操作都不会把新点与此前解码过的所有点作比较。

### 5.1 分布式推理示例

设想若干激光雷达节点观测一个场景的不同部分。训练好的编码器被复制到每个节点, 所有节点在同一参考系中以相同的归一化表达坐标。节点  $d$  把它的本地分块编码为  $\{\mathbf{s}_f^{(d)}, \mathbf{s}_m^{(d)}, \mathbf{s}_b^{(d)}\}$ , 随后即可删除本地特征。中继可以对这些三元组的任意子集求和; 服务器再把中继的输出相加, 得到式 (15)。到达顺序与通信树的形状都不影响实数运算下的结果。无论本地分块包含数百个点还是代表一段长的累计流, 每个被传输的三元组都是同样的 255 KiB。

对分类, 服务器把合并后的三元组直接送入状态行读出。新增一台设备或一帧只是再加一个三元组来改变类别证据, 无需重新解码并池化更早的点云。对分割, 服务器把坐标  $\mathbf{q}$  送入三次矩阵读取。查询可以是选定观测点的坐标、操作员指定的区域, 或规则网格上的位置; 读取时只需要这些坐标可用。由于式 (6) 对  $\mathbf{q}$  光滑依赖, 相邻查询会得到平滑变化的状态证据, 包括在未被显式观测的坐标处。当前实验只在观测到的 ShapeNetPart 点上评估标签, 因此在新坐标上的精度仍是一个开放的经验问题。

### 5.2 固定状态保留了什么、丢弃了什么

固定状态是摘要, 而不是被压缩的点档案。它们足以求值所学到的读出头, 但并不是为重建每一个原始观测而设计的。两个点集可能产生相近的矩表, 尤其当许多空间地址相互重叠

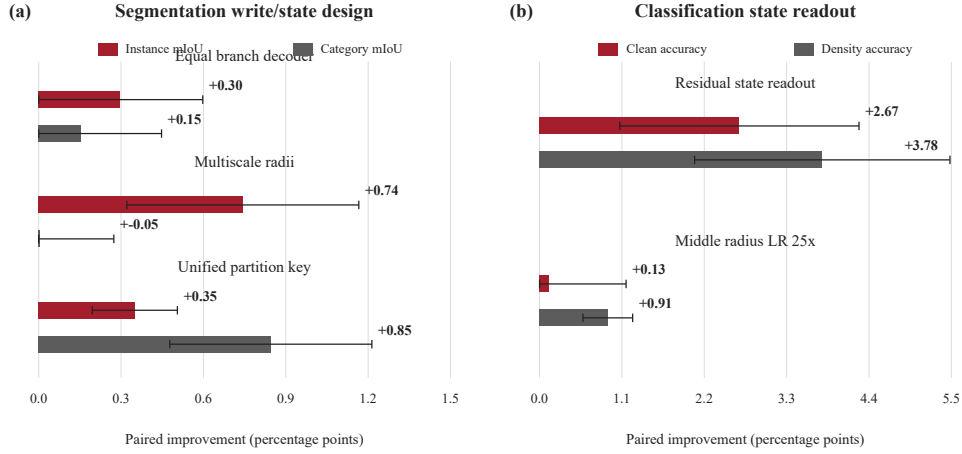


图 5: 三种子、5 个 epoch 的架构筛选。柱表示配对均值变化，误差棒表示总体标准差。分割部分检验等宽分支解码器、不同半径初始化，以及统一的归一化空间地址；分类部分检验残差式与拼接式分支摘要，以及中尺度半径学习率。

时。学习到的中心、半径、内容网络与二阶项决定了哪些区分能在这种压缩中存活。这既解释了有界内存的收益，也解释了与“保留或重建详细点关系”的邻域方法之间仍存在的精度差距。

更确切地说，一个点一旦被写入，状态就是之后任何阶段唯一能看到的東西，因此没有任何解码器能够恢复写入时已经丢弃的区分。**状态而不是读出，才是该架构真正起约束作用的压缩瓶颈。**筛选实验直接支持这一点：改变分割解码器只使实例 mIoU 变化  $0.30 \pm 0.30$  个百分点，改变其宽度或共享方式也只产生同量级的亚个百分点变化；而第 4.2 节中改变状态配置则使其变化超过一个百分点。这正是本设计把结构预算花在“点如何被写入”——归一化地址、二阶值以及采用若干并行状态——而不是花在“状态如何被解码”上的原因，也解释了为什么加宽主状态无法替代并行结构。

PointNet++ 在观测点周围构建邻域，PointNet 保留逐通道极值，而 PointProgrammerNet 保留有限的矩表。对短点云而言状态开销还超过裸 XYZ 存储；其收益出现在有界通信、合并或长流足以抵偿这一固定成本的场景。分布式等价性要求完全相同的预处理与共享坐标系，分割仍需为每个被请求的坐标读取一次状态。评估仅限于 XYZ 输入、每个任务一个数据集、一块计时 GPU 与本地受控基线。未观测坐标上的精度、时序校准、更多属性以及 delta 自适应仍是未来工作。

## 6 结论

PointProgrammerNet 用固定的可加矩阵对点云作摘要。连续的键与二阶值支持无序写入、精确的分布式合并、移除、衰减，以及代价与历史长度无关的更新。若干并行状态所编码的可用结构多于同等参数预算下的单一状态，而真正起约束作用的压缩瓶颈是状态本身而非读出。坐标条件化的分割与纯状态的分类都在不对解码点做池化的情况下工作。该模型为“从有界流式状态出发的集中式或分布式点云推理”提供了一个紧凑且可检验的设计，其相对层次化基线

的精度代价由实验予以量化。

## 声明

**利益冲突。**作者声明不存在与本职工作相关的经济或非经济利益冲突。

**资助。**本研究未接受任何资助。

**作者贡献。**Dian Yu: 概念构思、方法学、软件、验证、形式分析、研究实施、数据管理、可视化、初稿撰写、审阅与编辑。

**数据可用性。**ShapeNetPart 与 ModelNet40 均为公开数据集, 可从本文引用的原始来源获取。所发布的最小代码包含 341,909 参数的分割器与 529,426 参数的分类器、数据集加载器、训练循环、状态运算检查以及精确的三种子参考数值; `CODE_REPRODUCTION.md` 说明了环境、数据集布局、命令与流式 API。可加性测试应在评估模式下运行, 因为此时 BatchNorm 使用命题 1 所假设的固定滑动统计量。

## 参考文献

- [1] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation. In *CVPR*, pages 652–660, 2017.
- [2] Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J. Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. In *NeurIPS*, volume 30, pages 5099–5108, 2017.
- [3] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander J. Smola. Deep sets. In *NeurIPS*, volume 30, pages 3391–3401, 2017.
- [4] Hervé Jégou, Matthijs Douze, Cordelia Schmid, and Patrick Pérez. Aggregating local descriptors into a compact image representation. In *CVPR*, pages 3304–3311, 2010.
- [5] Relja Arandjelović, Petr Gronat, Akihiko Torii, Tomas Pajdla, and Josef Sivic. NetVLAD: CNN architecture for weakly supervised place recognition. In *CVPR*, pages 5297–5307, 2016.
- [6] Mikaela Angelina Uy and Gim Hee Lee. PointNetVLAD: Deep point cloud based retrieval for large-scale place recognition. In *CVPR*, pages 4470–4479, 2018.
- [7] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *ICML*, volume 97 of *PMLR*, pages 3744–3753, 2019.
- [8] Andrew Jaegle, Sebastian Borgeaud, Jean-Baptiste Alayrac, Carl Doersch, Catalin Ionescu, David Ding, Skanda Koppula, Daniel Zoran, Andrew Brock, Evan Shelhamer,

- Olivier Hénaff, Matthew M. Botvinick, Andrew Zisserman, Oriol Vinyals, and João Carreira. Perceiver IO: A general architecture for structured inputs and outputs. In *ICLR*, 2022.
- [9] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *ICML*, volume 119 of *PMLR*, pages 5156–5165, 2020.
- [10] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *ICML*, volume 139 of *PMLR*, pages 9355–9366, 2021.
- [11] Dingkan Liang, Xin Zhou, Wei Xu, Xingkui Zhu, Zhikang Zou, Xiaoqing Ye, Xiao Tan, and Xiang Bai. PointMamba: A simple state space model for point cloud analysis. In *NeurIPS*, 2024.
- [12] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 3d shapenets: A deep representation for volumetric shapes. In *CVPR*, pages 1912–1920, 2015.
- [13] Li Yi, Vladimir G. Kim, Duygu Ceylan, I-Chao Shen, Mengyan Yan, Hao Su, Cewu Lu, Qixing Huang, Alla Sheffer, and Leonidas Guibas. A scalable active framework for region annotation in 3d shape collections. *ACM Transactions on Graphics*, 35(6):210:1–210:12, 2016.
- [14] Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, Jianxiong Xiao, Li Yi, and Fisher Yu. Shapenet: An information-rich 3d model repository. *arXiv:1512.03012*, 2015.